

Prompt Engineering in Knowledge Engineering

Knowledge Representation and Knowledge Engineering

Michał Burda Michał Broś

AGH University of Krakow

May 28, 2026



Agenda

1. Intro & Motivation
2. Theoretical Foundations
3. Main Course: Methodology & Algorithms
4. Tools & Demo
5. Practical Application Examples
6. Evaluation & Critics
7. Sources & Literature

1. Motivation: Why is it worth studying?

- **The Shift:** We are moving from building ontologies by hand to using AI for help.
- **The Attraction:** Large Language Models (LLMs) have a lot of hidden knowledge. Prompt Engineering is how we get this knowledge out.
- **The Problem:** LLMs just guess words, but Knowledge Representation (KR) needs strict rules. Good prompts bridge this gap.

1. Simple Working Example

How changing a prompt changes the result:

Normal Prompt (User)

"Tell me about Albert Einstein and where he worked."

Result: A long text without structure.

Structured Prompt (Engineer)

"Extract facts from the text.
Format: (Subject, Predicate, Object)."

*Result: (Einstein, worksAt,
Institute_for_Advanced_Study)*

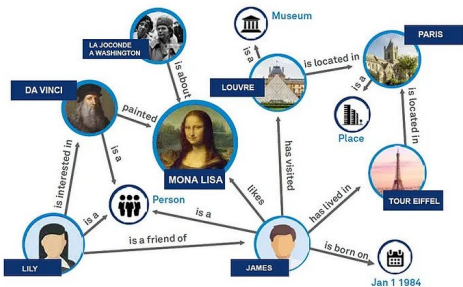
2. Theoretical Foundations & Definitions

- **Language Model (LLM):** A math function $P(w_t | w_1, \dots, w_{t-1})$ that guesses the next word based on previous words.
- **Context Window:** The short-term memory of the model. Everything the model needs to know now must fit here.
- **Hidden Knowledge Space:** The model stores meanings and connections inside its network as numbers (vectors).

2. Symbolic vs. Connectionist KR

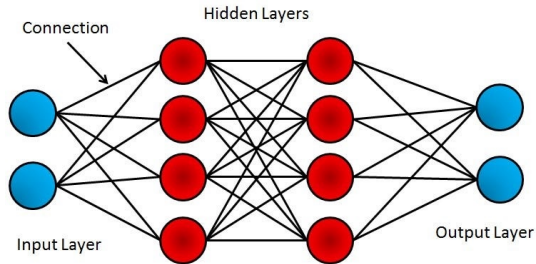
Classical (Graphs, Predicates)

- Strict rules & logic.
- 100% clear and sure.



LLM (Tensors, Probabilities)

- Hidden vector data.
- Guesses words based on math.



3. Methodology: Step-by-Step Reasoning (CoT)

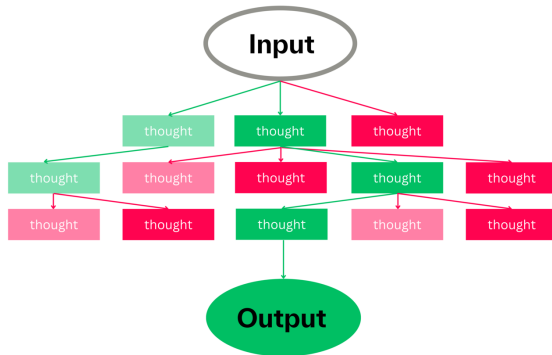
Chain-of-Thought (CoT)

- **The Idea:** Ask the model to show its thinking step-by-step before it gives the final answer.
- **How it works:** It gives the model more "time" (more words) to think about complex logic.
- **In KR:** This is very important to avoid logical mistakes.

3. Methodology: Tree & Graph of Thoughts

Advanced prompts using classic search methods:

- **Tree of Thoughts (ToT):** Checks many paths. If it makes a mistake, it can go back (*backtracking*).
- **Graph of Thoughts (GoT):** Connects different good ideas together to find the best answer.



Tree of Thoughts-Prompting
(ToT)

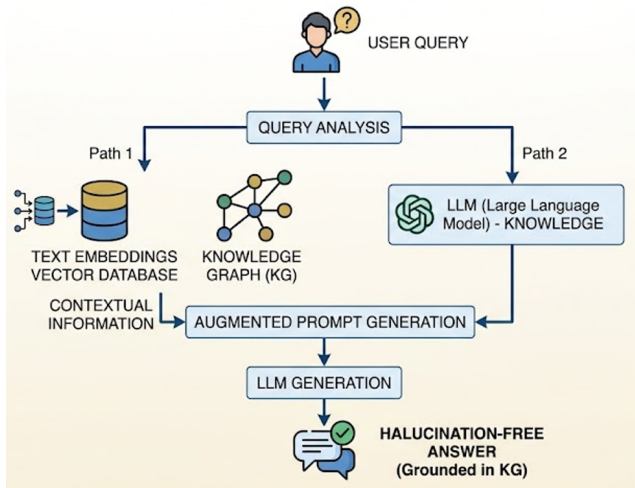
3. Methodology: Graph RAG

Retrieval-Augmented Generation (RAG) with Graphs

- **Algorithm:**

1. Read prompt → 2. Search Graph DB → 3. Get facts → 4. Put facts into the Prompt → 5. Generate answer.

- **Goal:** Base the answer on real facts from the graph, not just the model's memory.



4. Tools & Ecosystem

We use programming tools, not just chat windows:

- **LangChain / LlamaIndex:** Python libraries to build AI apps.
- **Ontology Tools:** Protégé, RDFlib (Python).
- **Workflow:** A Python script builds the prompt → calls the API (like OpenAI) → reads the JSON → updates the Database.

4. Prompt Evolution: Step 1 and 2

Task: Get data from text.

"John Smith founded TechPol in 2010. He works with Anna, the lead engineer."

Step 1: Simple Prompt

Prompt: Extract people and relations from this text.

Output: "Sure! Here are the people: 1. John Smith - founder of..." (Bad for programming)

Step 2: Role & Format Prompt

Prompt: Act as a data tool. Extract relations as (Subject, Predicate, Object).

Output: (John Smith, founded, TechPol), (Anna, is, lead engineer) (Better, but still not perfect)

4. Prompt Evolution: The Final Form

Step 3: Strict Prompting for Knowledge Engineering

System Prompt: You are a strict Knowledge Graph tool.

Rules:

1. Output ONLY valid JSON. Do not write normal text.
2. Use ONLY these words: [founderOf, worksWith, leadEngineerAt].

User Prompt: {Text}

Perfect Output for Code:

```
{
  "triplets": [
    {"subject": "John_Smith", "predicate": "founderOf", "object": "TechPol"},
    {"subject": "John_Smith", "predicate": "worksWith", "object": "Anna"}
  ]
}
```

4. Live Demo: Jupyter Notebook

Hands-on Presentation

We will now show you a live Jupyter Notebook to see how Prompt Engineering works in Python code.

In this demo, we will show:

- ➊ Connecting to the API (e.g., OpenAI).
- ➋ Setting up the System Prompt in LangChain.
- ➌ Running the data extraction task.
- ➍ Checking the JSON output.

(Switching to IDE...)

5. Practical Problem Solving: Knowledge Extraction

Problem: You have 10,000 pages of medical text and want to build an OWL ontology.

- **Solution:** Few-Shot Prompting.
- **How:** We give the AI 3 good examples of text changed into (Subject, Predicate, Object) triplets.
- **Execution:** The model reads the rest of the pages and builds the ontology automatically.

5. Practical Problem Solving: Ontology Alignment

Problem: Two companies merge. Company A uses the word *"Client_ID"*, Company B uses *"Customer_Number"*.

- **Solution:** Use AI to match similar words.
- **Execution:** The AI compares definitions. It writes an `owl:equivalentClass` rule, meaning both words are the same thing.

6. Evaluation & Limitations

Problems with Prompt Engineering

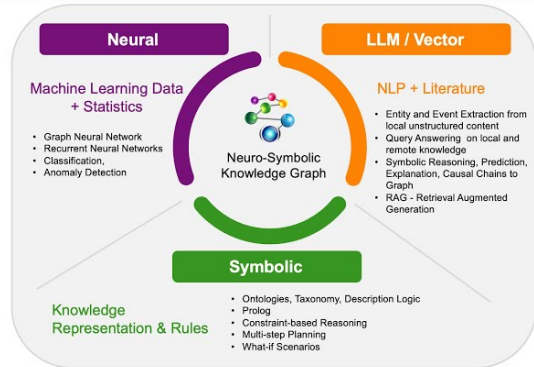
- **Fragility:** A bad comma or synonym can break the prompt (unlike strict SQL queries).
- **Hallucinations:** AI sometimes invents fake facts. It does not know when it is wrong.
- **Randomness:** The same prompt can give different answers if the settings change.

6. Perspectives: Neuro-Symbolic AI (SOTA)

The future is combining two different systems:

- **System 1 (Neural):** The LLM is great at understanding natural human language.
- **System 2 (Symbolic):** The Knowledge Graph keeps the facts strictly correct.

Neuro-Symbolic AI Knowledge Graph Architecture



7. Full Repository & Codes

Project Repository

The prompts and Python code we used today are available in our repository.

- **GitHub Link:** <https://github.com/MichalBurda4/KRKE-Prompt-Engineering>
- **Stack:** Python, LangChain, OpenAI API, RDFlib, Jupyter.

7. Literature & Sources



Wei, J. et al. (2022).

Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.
NeurIPS.



Yao, S. et al. (2023).

Tree of Thoughts: Deliberate Problem Solving with Large Language Models.
arXiv preprint.



Lewis, P. et al. (2020).

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.
NeurIPS.



Pan, S. et al. (2023).

Unifying Large Language Models and Knowledge Graphs.
IEEE Transactions on Knowledge and Data Engineering.